

Prototype-Based Federated Learning Methodologies

1. Overview of Prototype-Based Federated Learning (PFL)

Prototype-based Federated Learning (PFL) represents a fundamental paradigm shift in distributed optimization. Moving beyond the traditional synchronization of high-dimensional model parameters (e.g., FedAvg), PFL architectures communicate "prototypes"—compact, representative embeddings or class-specific centroids—within a shared latent space. This approach effectively decouples global knowledge from the specificities of local model architectures, enabling a unified representational framework across a heterogeneous federation. The primary motivations for this paradigm are three-fold:

- **Reduction of Communication Overhead:** By transmitting lightweight class representatives instead of the millions of parameters typical of deep neural networks, PFL minimizes bandwidth requirements and enables scaling in resource-constrained environments.
- **Preservation of Data Privacy and Intellectual Property (IP):** Sharing compact prototypes rather than raw gradients or weights provides a "black-box" layer of protection. This architecture safeguards sensitive local model weights and architectures (IP) while preventing data inversion attacks that target full parameter updates.
- **Mitigation of Data and Model Heterogeneity:** PFL facilitates seamless collaboration between clients utilizing entirely different model architectures (Model Heterogeneity) and disparate data sources (Data Heterogeneity) by focusing on feature-level alignment rather than parameter-level consensus.

2. Taxonomy of Heterogeneity Challenges

Technical innovation in PFL is driven by the need to resolve specific mathematical failures inherent in distributed non-IID settings:

- **Feature Drift:** A distribution shift where samples of the identical class exhibit different feature distributions across clients due to variations in sensors, environments, or collection protocols (e.g., medical imaging from different hospitals).
- **Client Drift:** The divergence of local models from the global objective during isolated training.
- **Prototype Drift:** The temporal divergence where local prototypes are pulled toward client-specific biases, losing global semantic consistency.
- **Model Heterogeneity:** The inability to aggregate parameters when clients utilize diverse architectures within the same federation.
- **Prototype Margin Shrink:** The degradation of global prototype separability caused by "naive" weighted-averaging. When well-separated client prototypes are averaged without regarding their specific inter-class margins, the resulting global prototypes often exhibit reduced separability, diminishing the global model's discriminative power.

3. Thematic Categorization of Frameworks

Theme A: Advanced Clustering & Domain-Aware Aggregation

This theme utilizes clustering and domain-aware fusion to capture multi-domain variance.

- **FPL (Federated Prototypes Learning):**
 - **Problem Solved: Domain Shift:** When data is distributed across distinct domains, naively averaging local prototypes into a single global signal destroys domain diversity and creates a bias toward majority domains. The private models suffer from poor generalization and fail to discriminate properly on other domains because the global signal is insufficient to depict diverse domain knowledge.
 - To solve this, the server utilizes a parameter-free clustering algorithm (FINCH) to group local prototypes into multiple diverse "Cluster Prototypes," which preserve rich, domain-specific semantic characteristics. To ensure stable optimization, FPL then averages these cluster prototypes to create "Unbiased Prototypes," preventing the objective direction from being skewed by dominant domains. Local clients then optimize using Cluster Prototypes Contrastive Learning (CPCL) and Unbiased Prototypes Consistent Regularization (UPCR).
 - FPL is specifically designed to address imbalances in client distribution across domains, aiming to neutralize the skewed influence of domains with more clients on the global model training.
- **FedPLVM (Federated Prototype Learning with Variance Mitigation):**
 - **Problem Solved: Cross-Domain Representation Variance (Domain Shift):** Existing methods create the same number of prototypes for each client, ignoring the unequal learning difficulties between "easy" domains (where features cluster neatly) and "hard" domains (where features are scattered and overlap). This leads to cross-domain performance gaps and disparities across clients.
 - **variance-aware dual-level prototype clustering** mechanism. At the first level (client-side), local features are clustered to generate multiple local prototypes, which allows hard domains with scattered distributions to be accurately represented by a larger, dynamic number of prototypes. At the second level (server-side), these local prototypes are re-clustered into a compact set of global prototypes, significantly reducing communication overhead and hiding client-specific private attributes. To regulate these diverse clusters, FedPLVM introduces an α -sparsity prototype loss that strategically increases inter-class sparsity while adjusting for intra-class similarities.
- ★ **FPL uses single-level, server-side clustering.** The server collects one prototype per class from each client and applies the FINCH clustering algorithm to group them into multiple "Cluster Prototypes." To ensure a

stable and fair target, the server averages these cluster prototypes to create "**Unbiased Prototypes**".

★ **FedPLVM uses dual-level clustering (client-side and server-side).**

- **First-Level (Client-Side):** Because "hard" domains have scattered feature distributions, clients generate multiple local clustered prototypes per class (rather than just one) to accurately capture this variance.
- **Second-Level (Server-Side):** To prevent the massive assortment of local prototypes from clogging communication bandwidth and leaking privacy, the server aggregates and re-clusters them into a refined, compact set of global prototypes

● **FedDAP (Domain-Aware Prototype Learning):**

- **Problem Solved: Domain Shift** Forcing all clients to align their features to a single, domain-agnostic global prototype causes "semantic conflict" and "semantic dilution". For example, a "dog" in a photo domain has vastly different visual characteristics than a "dog" in a sketch domain; forcing them into the exact same prototype representation weakens the model's generalization ability.
- Unlike clustering approaches, FedDAP explicitly constructs distinct **domain-specific global prototypes** for each (class, domain) pair. Clients upload their local prototypes tagged with explicit Domain Identifiers. The server aggregates these using a **cosine similarity-weighted fusion mechanism**, assigning higher weights to client prototypes that exhibit strong semantic consistency with their style peers, thereby filtering out anomalies. Local training utilizes a Dual Prototype Alignment Strategy: **Domain-Consistent Prototype Alignment** forces local features to strictly align with their specific class-and-domain prototype, while **Cross-Domain Prototype Contrastive Learning** forces local features to push away from different classes across all domains, enabling domain-invariant semantics.
- **Cosine Similarity-Weighted Fusion (Server Aggregation):** Even within the same domain (e.g., two clients both sending "Photo" prototypes), some clients might have poorly trained models or noisy data. The server groups all incoming prototypes strictly by their (class, domain) matches. It then computes a **consistency score for each client's prototype by measuring its cosine similarity against all other client prototypes in that exact same group**. An attention score (α) is assigned based on this similarity. **Prototypes that closely match their peers get high weights, while anomalies get low weights.** This attention-based fusion mathematically isolates and suppresses bad local updates.

3. Dual Prototype Alignment Strategy (Local Client Training) Clients use the server's grid of domain-specific global prototypes to train their local feature extractors along two complementary axes:

- **Axis A: Domain-Consistent Alignment (Intra-Domain):** Forces local features to strictly align with the global prototype of the **same class and same domain**. Uses **cosine similarity** (angle) instead of standard Euclidean distance.
- **Axis B: Cross-Domain Contrastive Learning:** Uses a contrastive loss function to expose the local model to other domains.
 - **Positive Set:** Prototypes of the **same class from other domains** (e.g., pulling a sketch-dog toward photo-dogs).
 - **Negative Set:** Prototypes of **different classes from other domains** (e.g., pushing a sketch-dog away from photo-birds).
 - **Benefit:** Pulling the same classes together teaches the model **domain-invariant semantics** (the core traits of a class regardless of style). Pushing different classes apart sharpens decision boundaries, preventing misclassification when the domain shifts.

Theme B: Trainable & Learnable Prototypes

This theme treats prototypes as actively trainable parameters optimized via gradient descent. This paradigm enhances prototype separability and semantic robustness against client drift.

- **FedTGP (Trainable Global Prototypes):**
 - **Problem Solved: Data and Model Heterogeneity ("Prototype Margin Shrink"):** In settings where clients have different model architectures and data distributions, performing a simple weighted average of client prototypes generates uninformative global prototypes with very small class margins. This "prototype margin shrink" diminishes the ability of the model to separate different classes effectively.
 - FedTGP decouples prototype learning from client data by creating Trainable Global Prototypes (TGP) on the server: The server decouples prototype generation from client data by establishing the global prototypes as actively trainable parameters. They are optimized dynamically on the server rather than just being statically computed empirical means.
 - **Contrastive Optimization:** The server optimizes these global prototypes using a contrastive loss objective. This process actively pushes prototypes of the same class together (ensuring semantic alignment) while forcefully pushing prototypes of different classes apart to create clear separation

- **Adaptive Margin Sizing:** Enforcing a rigidly large separation gap too early in training can degrade performance because the clients' feature extractors are initially weak. To address this, the ACL mechanism uses an adaptive margin. In each communication round, the server measures the maximum natural distance actually achieved between client classes and uses that exact distance as the target separation margin. This ensures the model is always aiming for the best achievable separability at any given stage of training
- **FedHPro (Federated Hyper-Prototype Learning):**
 - **Problem Solved: Data Heterogeneity (Semantic/Client Drift under Domain, Label, and Quantity Skew):** Updating global prototypes by simply averaging biased local representations causes "semantic drift." Because the local prototypes are collected from highly diverse and skewed data preferences, the resulting global signal becomes misaligned and fails to accurately reflect the true class centers.
 - **Hyper-Prototypes:** Learnable global anchors optimized via Gradient Matching. Clients do not send raw prototypes; instead, they send class-wise gradients distilled from real local samples. The server mathematically adjusts its Hyper-Prototypes so that their optimization trajectory perfectly mimics these exact gradients, effectively learning from the entire data distribution without accessing raw data. Clients then use these hyper-prototypes to perform Hyper-Prototype Contrastive Learning (HPCL) and Hyper-Prototype Alignment Learning (HPAL) to promote inter-class separability and strict intra-class uniformity.
- **FedProIn:**
 - **Problem Solved: Client Drift (Specifically Feature Drift and Prototype Drift):** Statistical heterogeneity causes distinct models to map identical clinical concepts to misaligned regions in the feature space (Feature Drift). Concurrently, local prototypes get pulled toward client-specific biases, causing them to deviate from a generalized global representation (Prototype Drift).
 - **Learnable Prototypes:** Instead of treating prototypes as static, temporary averages of features, FedProIn uses **Learnable Prototypes (LPs)**—treating them as actively trainable model parameters that are optimized directly via backpropagation.
 - **1. Local Client Training (Dual Loss Strategy)** To fix drift at the client level, FedProIn applies two specific loss functions during local training:
 - **Feature Divergence Loss (FDL):** The client forces its active local features to align with the features produced by a frozen copy of the global encoder.
 - Benefit: Acts as an anchor to stop the local feature space from wildly drifting away from the global consensus (solves Feature Drift).

- **Prototype Contrastive Loss (PCL):** A contrastive mechanism that pulls active **local prototypes toward the generalized global prototypes, while aggressively pushing them away from the client's own biased local prototypes from the previous round.**
 - Benefit: A strict margin control ensures prototypes stay closer to the global template than to local historical biases, preserving their universal clinical meaning (solves Prototype Drift)
- **FDL operates at the Instance/Feature Level:** It acts directly on the raw feature embeddings extracted from individual data samples (like a single medical image).
- **PCL operates at the Class/Prototype Level:** It acts on the "Learnable Prototypes," which are the overarching mathematical templates representing an entire class (e.g., the general concept of a specific disease). It does not look at individual images; it looks at the overall class centroids

2. Server-Side Aggregation (Normalized Influence Aggregation) When clients send their updates back to the server, FedProIn splits the aggregation into two protocols:

- **Feature Encoder:** **Averaged using standard FedAvg** (weighted by client dataset size).
- **Normalized Influence Aggregation (NIA) for Prototypes:** Standard averaging fails for multiple prototypes because rare disease prototypes might be entirely unused by certain hospitals.
 - The Mechanism: **Clients keep a running tally (an "influence matrix") of exactly how many times each class prototype was actually the closest match for a training sample.**
 - The Benefit: **The server averages prototypes proportionally based on this real-time usage count rather than total dataset size. This suppresses noise from unutilized prototypes while perfectly preserving the highly specialized insights from the few clinics that actually possess rare disease samples**

Theme C: Geometric & Structural Prototype Adjustments

This theme focuses on manipulating the strict geometric layout of prototypes in the latent space. By explicitly controlling the spatial configuration of the global targets, these methods counteract representation collapse caused by non-IID class imbalances.

- **ProtoNorm (Prototype Normalization):**
 - **Problem Solved: Data and Model Heterogeneity (Suboptimal Prototype Separation):** In Prototype-Based Federated Learning, the spatial separation of global prototypes is completely dictated by the

empirical averaging of local data. This leads to suboptimal angular and spatial separation between classes in the latent space, severely limiting the discriminative power of the global model.

- ProtoNorm resolves this by sequentially applying Prototype Alignment (PA) and Prototype Upscaling (PU). Inspired by the Thomson problem in classical physics (distributing electrons on a sphere), the **PA** algorithm uses **gradient ascent on the server to distribute global prototypes uniformly across a unit hypersphere, thereby maximizing angular separation**. Because neural networks can get stuck in suboptimal local minima if global prototypes remain small unit vectors, the **PU method subsequently scales these aligned prototypes outward by a magnitude hyperparameter (γ) to vastly improve spatial separation in Euclidean space**.
- **FedNH (Non-parametric Head):**
 - **Problem Solved: Data Heterogeneity with Class Imbalance (Minority Class Collapse):** When clients have highly imbalanced data, the representations of minority classes are frequently overridden or collapse into the overlapping space of dominant, majority classes. This ruins the local model's personalized performance and generalization on underrepresented classes.
 - FedNH enforces an inductive geometric constraint known as **Prototype Uniformity**. **Before training, the server initializes fixed, maximally separated class prototypes uniformly on a unit hypersphere**. During local training, the classification head is frozen, and clients only train their feature extractors to hit these rigid targets. To restore real-world class relationships, FedNH applies **Semantic Infusion**: the server performs a non-parametric moving average update, smoothly blending the rigid, uniform geometric prototypes with the actual data-driven, semantic local representations aggregated from clients.

Theme D: Hybrid Architectures & Adversarial Collaboration

These methods merge prototype-based learning with specialized structural adaptations—such as adversarial networks or frozen backbone architectures—to unify disparate feature spaces or drastically reduce communication overhead.

- **FedPall:**
 - **Problem Solved: Feature Drift:** A specific form of heterogeneity where samples of the exact same class exhibit vastly different feature distributions across clients.
 - FedPall combines prototype contrastive learning with an adversarial client-server dynamic. The server trains a **global Amplifier network** to predict the origin client ID of incoming features. Clients utilize KL Divergence loss against a frozen copy of this Amplifier to actively scrub

domain-specific markers from their feature extractors, effectively "confusing" the Amplifier and forcing all features into a unified distribution. Finally, to train a robust global classifier, clients linearly blend their aligned local features with global prototypes and apply a **Bernoulli masking mechanism** to secure the data before transmitting it to the server.

- **FedAdapterProto:**

- **Problem Solved: Statistical Heterogeneity and Communication Overhead** in Medical Imaging: Sharing entire large-scale medical models is communication-heavy, while statistical heterogeneity across different clinical sites severely degrades performance. Standard prototype approaches reduce communication but fail to adapt the local feature extractors for individual site generalization
- This framework uniquely combines prototype guidance with **Lightweight Adapters**. To minimize communication bandwidth when dealing with heterogeneous clinical sites, FedAdapterProto strictly freezes the heavy, primary feature-extracting backbone on all client models. The only parameters synchronized with the server are small, low-dimensional adapter modules (which act as compressed surrogates to guide model generalization) and the prototype layers. Here, the prototypes not only facilitate federated aggregation but also provide inherent interpretability by acting as reference templates for characteristic parts of the input classes.